S. No.	Date	Title	Page No.	Sign / Remarks
1.	22/7/26	Arrays of structure creation, insertion deletion, traversal, Sorting.		
2.	5/8/26	Student details.		
3.	12/8/26	Display value and address using pointers		
4.	19/8/26	C program for building a dynamic Employee Record system using structure pointers.		
5.	26/8/26	stack using Array.		
6.	2/9/26	Simple Queue		
7.	9/9/26	single linked List.		
8.	16/9/26	Circular Queue		
9.	7/10/26	Binary search tree.		
10.	14/10/26	graph search algorithm (Bfs & Dfs)		
11.	12/10/26	Implemention of Bubble, ~~Sort~~ Selection and Heap sorts.		
12.	21/10/26	Implementation of Hashing with collision Resolution using Linear probing		

1. Array creation

Aim : To create array creation elements

Algorithm :

1. Start
2. Declare an integers array of size
3. Read 5-elements and store them in array
4. Display the array elements
5. Stop

Program :

```
#include <stdio.h>
    int main()
{
 int arr[5];
 int i;
Printf("Enter 5-elements :\n");
    for (i=0; i<5; i++)
    {
     scanf("%d", &arr[i]);
     }
     return 0;
     }
```

Sample Input :

Enter 5 - elements :

10
20
30
40
50

Sample output :

Array elements are :

10 20 30 40 50

Result :

the program for array displaying was executed and verified

2. Array insertion

Aim :

insert an element into a given position in an array

Algorithm :

1. Start
2. Read the number of array
3. Read the array element
4. Position & element to be inserted
5. Shift the element position from right from the insertion position
6. Insert the new element at specified position
7. Display updated array
8. Stop

Program :

```
#include <stdio.h>
int main() {
    int a[10], n, i, pos, item;
    printf("Enter the n");
    scanf("%d", &n);
    for (i=0; i<n; i++);
        scanf("%d", &a[i]);
```

```
printf("Enter a pos");
scanf("%d", &pos);
printf("Enter a element");
scanf("%d", &item);
for(i=n; i>=pos; i--)
    a[i] = a[i-1];
a[pos-1] = item;
for(i=0; i<n; i++)
    printf("%d", a[i]);
return 0;
}
```

Sample input:

5
10 20 30 40 50
3
25

Sample output:

10 20 30 40 50 25

Result:

The program for displaying the array is successful

3. Array Declaration

Aim : Deletion of the element from an array

Algorithm :

1. Start
2. Read the size of array : n
3. Read the array element
4. Read position of element to be deleted
5. Shift all elements after the deleted position one place to left
6. Decrease array size by 1
7. Display updated array
8. Stop

Program :

```
#include <stdio.h>
int main()
{
int a[10], n, i, pos;
printf("Enter the nth element");
scanf("%d", &n);
for (i=0; i<n ; i++)
scanf("%d", &a[i]);
```

Sample input :

Enter the number of elements : 5

Enter array elements :

10 20 30 40 50

Enter position to delete : 30

Sample output :

Array after deletion :

10 20 40 50

Result :

Insertion at a specific index was created successfully

4. Array Transversal

Aim : Transversal of an array elements

Algorithm :

1. Start
2. Read the no of elements 'n'
3. Read array elements.
4. Transversal array form 1st element to the last element using for-loop
5. Display each element
6. Stop

Program :

```
#include <stdio.h>
int main()
{
int a[10], n, i, j
printf("Enter n elements");
scanf("%d", &n);
for (i=0, i<n, i++)
    scanf("%d", &a[i])
    return 0;
}
```

Sample input:

Enter the number elements : 5

Array elements :

10 20 30 40 50

Sample output:

Array : Elements are :

10 20 30 40 50

Result :

Transversal of Array element is executed successfully

Aim :

Algorithm :

1.

2.

3.

4.

5.

6.

7.

8.

Program :

5. Array searching

Aim: Searching in an array elements.

Algorithm:

1. Start
2. Read the no of elements
3. Read the array element
4. Read the element to be searched
5. Compare the element with each array element.
6. If found, Print "Element found".
7. Else Print "Element not found"
8. Stop

Program:

```
#include <stdio.h>
int main()
{
int a[10], n, i, key;
printf("Enter n elements");
scanf("%d", &a[i]);
for (i=0, i<n, i++);
scanf("%d", &a[i]);
Printf("Enter the key elements");
scanf("%d", &key);
for(i=0 i<n; i++)
{
if (a[i] == key) {
Printf("Element found");
return ;
}}
Printf("Element not found");
return 0; }
```

Sample input :

Enter no of elements : 5

Enter array elements :

20 40 60 80 30

Sample output :

Array Elements are :

20 40 60 80 30

Enter element to be search : 60

Element found.

Result :

Searching in an array element executed successfully

6. C-program for building a Dynamic Employee record system using structure pointers

Program:

```
#include <stdio.h>
#include <string.h>
#define MAX_EMPLOYEES 100

struct Employee {
    int id;
    char name[50];
    float salary;
};

struct Employee *emp[MAX_EMPLOYEES]; /* array of pointers to Employee */
int count = 0; /* how many employees stored so far */

void addEmployee() {
    if (count >= MAX_EMPLOYEES) {
        printf("Employee List is full\n");
        return;
    }
    struct Employee *e = (struct Employee *) malloc(sizeof(struct Employee));
    printf("Enter ID: ");
    scanf("%d", &e->id);
    printf("Enter name: ");
    scanf("%s", e->name);
```

```
scanf("%f", &c->salary);
emp[count] = c;
count++;
printf("Employee added!\n\n");
}
void displayEmployees() {
if (count == 0) {
printf("No employees to show\n\n");
return;
}
printf("\nID\tname\tsalary\n");
for (int i=0; i<count; i++) {
printf("%d\t%s\t%.2f\n", emp[i]->
emp[i]->name,
emp[i]->salary);
}
printf("\n");
}
void searchEmployee() {
int id;
printf("Enter ID to search:");
scanf("%d", &id);
for (int i=0; i<count; i++) {
if (emp[i]->id == id) {
printf("Found: %s, salary: %.2f\n",
emp[i]->name, emp[i]->
return;
}
```

```
printf("Employee not found.\n\n");
}
void deleteEmployee() {
  int id;
printf("Enter ID to delete:");
  scanf("%d", &id);
  for (int i=0; i<count; i++) {
  if (Emp[i]->id == id) {
free(Emp[i]); /* free that employee's memory */
Emp[i] = Emp[count-1]; /* move last employee into this
                          slot */
  count--;
  printf("Employee deleted.\n\n");
  return;
}
}
printf("Employee not found.\n\n");
}
void freeAll() {
  for (int i=0; i<count; i++) {
    free(Emp[i]);
  }
  count = 0;
}
int main() {
  int choice;
  do {
  printf("1. Add 2. Display 3. Search 4. Delete
                    5. Exit\n");
```

campus
Size : 31 cm x 19 cm
Pages : 72 80 92 112 12
M.R.P. : ₹40 ₹45 ₹50 ₹55 ₹6
Pages inclusive of cover & printed matter
RP inclusive of all taxes
w.mybookfactory

```
printf("choice:");
scanf("%d", &choice);
switch(choice){
case 1: add employee(); break;
case 2: display Employee(); break;
case 3: search Employee(); break;
case 4: Delete Employee(); break;
case 5: free All();
printf("Bye!\n"); break;
default: printf("invalid choice.\n");
}
} while(choice != 5);
return 0;
}
```

Experiment No-7:

Implementation of single Linked List (Creation Deletion & Traversal)

Aim: To write a C-Program to implement a singly Link List & perform insertion, deletion & Traversal

Algorithm:

1. Start the program & define a structure node contain an integer data field & a pointer next. Initialize head to NULL
2. Insert at beginning: create a new node, set it's next to head & make head point to it. Traverse to the last node & Link it to new node
3. Delete at beginning: Store head in a temporary ptr, move head to head → next & free temporary node.
4. Delete at End: traverse to the node just before given position, Link it to node after one being deleted
5. Delete at position: traverse to the node just before given position Link it to node until null reached
6. Display a menu with all operations & select it until the user choose exit
7. Stop the program

campus
Size : 31 cm x 19 cm
Pages : 72 80 92
M.R.P. : ₹40 ₹45 ₹50
ages inclusive of cover & prin
RP inclusive of all taxes
w.mybookfac

Program :

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node * next;
};
struct node * head = NULL;
void insertBeginning (int val) {
    struct node * new node = (struct node *) malloc
        (sizeof (struct node));
    new node -> data = val;
    new node -> next = head;
    head = new node;
}
void insertEnd (int val) {
    struct node * new node = (struct node *) malloc
        (sizeof (struct node));
    new node -> data = val;
    new node -> next = NULL;
    if (head == NULL) {
        head = new node;
        return;
    }
    struct node * temp = head;
    while (temp -> next != NULL)
        temp = temp -> next;
    temp -> next = new node;
```

```
void insert position (int val, int Pos) {
    if (Pos == 0) {
    insert Begining.(val);
    return;
    }
struct node * new node = (struct node *) malloc (size of
                                    (struct node));
    new node -> data = val;
    struct node * temp = head;
    int i;
for(i=0; i< pos-1 && temp != null; i++)
    temp = temp -> next;
    if (temp == null) {
    printf("invalid position \n");
    free (new node);
    return;
    }
new node -> next = temp -> next
temp -> next = new node;
}
void dlet Begining.() {
    if (head == null) {
    print ("List is empty.\n");
    return;
    }
    struct node * node = head;
```

```
void deleteEnd(){
    if(head == NULL){
        printf("List is empty.\n");
    }
    if(head->next == NULL){
        free(head);
        head = NULL;
        return;
    }
    struct Node *temp = head;
    while(temp->next->next != NULL)
        temp = temp->next;
    free(temp->next);
    temp->next = NULL;
}
void deletePosition(int Pos){
    if(Pos == 0){
        deleteBeginning();
        return;
    }
    struct node *temp = head;
    int i;
    for(i=0; i<Pos-1 && temp != NULL; i++)
        temp = temp->next;
    if(temp == NULL || temp->next == NULL)
        printf("invalid position.\n");
        return;
    }
```

```
struct Node * toDelete = temp->next;
    temp->next = toDelete->next;
    free(toDelete);
}
void traverse() {
    struct Node *temp = head;
    if(head == NULL) {
    printf("List is empty.\n");
      return;
    }
    printf("Linked List:\n");
    while(temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}
int main() {
    int choice, val, pos;
    do {
        printf("\n---Singly Linked List
                menu---\n");
printf("1. Insert at Beginning\n2. Insert
    at position\n3. Insert at End\n");
printf("4. Delete at Beginning\n5. Delete at
    position\n6. Delete at End\n");
```

```
printf("7. Traverse In 8. Exit In");
scanf("%d", &choice)
switch (choice) {

case -i:
    Printf("Enter value :::");
    scanf("%d", &val);
    insert Beginning (val);
    Break;

case -ii:
    printf("Enter value & position (0-indexed):");
    scanf("%d %d", &val, &pos);
    insert position (val, pos);
    break;

case -iii:
    Print f("Enter value :");
    Scanf("%d", &val);
    insert End (val);
    Break;

case -iv:
    delete Beginning();
    break;

case -v:
    Printf("Enter Position (0-indexed):");
    Scanf("%d", &Pos);
    delete position (Pos);
    break;
```

case - vi:
delete
bra

case - vii:
trave
bra

case - viii:
P

Sample

1. in
2.
3.
4.
5.
6
7
8

Case - VI :

```
delete End ;
break ;
```

Case - VII :

```
traverse () ;
break
```

Case - VIII :

```
printf( " Existing Program \n") ;
break ;
default:
printf(" invalid choice \n");
}
} while (choice != 8);
return 0 ;
}
```

Sample output :

--- Singly Linked List menu ---

1. insert at beginning
2. insert at Position
3. insert at End
4. Delete at position
5. Delet at Beginning
6. Delete at End
7. Traverse
8. Exit

i. Enter your choice : 3

value : 10

ii. Enter your choice : 3

value : 20

iii. Enter your choice :

value : 30

iv) Enter your choice

Enter value & pos : 15 1

Enter your choice : 7

Linked List : 10 → 15 → 20 → 30 → Null

Enter your choice : 5

Linked position (0 - indexed) : 1

Enter your choice : 7

Linked List : 10 → 20 → 30 → Null

Enter your choice : 8

Exiting Program

Result :

Thus a program was written & executed successfully to implement insertion deletion & traversal operations on a singly Linked List

Implement Graph search Algorithms (BFS & DFS)

Aim:

To write a C Program to represent a graph using adjacency matrix and traverse it using Breadth-first search (BFS) and Depth-first search (DFS)

Algorithm:

1. Read the number of vertices and edges and construct the adjacency matrix representing the graph (mark adj[u][v] = adj[v][u] = 1 for each edge since the graph is undirected).

2. BFS. mark the starting vertex as visited and insert it into a queue; while the queue is not empty remove a vertex, Print it, and insert all its unvisited adjacent vertices into the queue making them as visited

3. DFS. mark the starting vertex as visited and Print it; recursively visit each unvisited adjacent vertex in the same manner

4. Display the sequence of vertices visited by BFS and by DFS starting from the given vertex

Program:

#include <
#define m
int adj[MAX]
int visited
int n; /*
void bfs(in
int qu
int
visitedloc
queue
Printf("
whi
gf

```
printf("\n");
}
void dfs util (int vertex) {
visited [vertex] = 1;
Printf ("%d", vertex);
for (int i=0; i<n; i++) {
if (adj [vertex][i] == 1 && !visited [i])
dfs util (i);
}
}
void dfs (int start) {
for (int i=0 ; i<n ; i++)
visited [i] = 0;
Printf ("DFS traversal :");
dfs util (start);
printf ("\n");
}
int main () {
int edges , u, v, start ;
Printf ("Enter number of vertices:");
Scanf ("%d", &edges);
Printf ("Enter edges (u,v) vertices numbers
'0 to %d :\n", n-1);
for (int i=0; i< edges ; i++) {
Scanf ("%d %d", &u, &v);
```

```
adj [u][v]
adj [v][u]
}
Printf ("Ent
Scanf (
bfs (
dfs (
return
}
Sample
input :
Ente
Er
E
Er
output
Result
DFS
ou
```

```
    adj[u][v] = 1;
    adj[v][u] = 1; /* undirected graph */
  }
printf("Enter starting vertex: ");
  scanf("%d", &start);
  bfs(start);
  dfs(start);
  return 0;
}
```

Sample input/output

input:

Enter number of vertices: 5

Enter number of edges: 5

Edges: 0-1, 0-2, 0-3, 2-4, 3-4

Enter starting vertex: 0

output:

BFS traversal: 0 1 2 3 4

DFS traversal: 0 1 3 4 2

Result: the C program to traverse a graph using BFS & DFS was implemented and executed successfully and the output was verified.

* Implementation and traversal (In order, Pre-order, Post-order) of Binary Search Tree

Program :

```
#include <stdio.h>
#include <stdlib.h>
struct node {
    int data;
    struct node * left * right;
};
struct node * (struct node (int value){
    struct node * new node = (struct node*) malloc(sizeof(stru)
    new node -> data = value;
    new node -> left = new node -> right = NULL;
    return new node;
}
struct node * insert (struct node * root, int value){
    if (root = NULL)
        return (create node (value);
    if (value < root -> data)
        root -> left = insert (root -> left, value);
    else if (value > root -> data)
        root -> right = insert (root -> right,
                                value);
```

Program :

```
#include <stdio.h>
#define MAX 20

int adj [MAX] [MAX];
int visited [MAX];
int n; /* number of vertices */

void bfs (int start) {
    int queue [MAX], front = 0, rear = 0;
    int visitedlocal [MAX] = {0};

visitedlocal [start] = 1;
queue [rear++] = start;

Printf ("BFS traversal :");
    while (front < rear) {
        int current = queue [front++];
        printf ("%d", current);

    for (int i = 0; i < n; i++) {

if (adj [current][i] == 1 && ! visited local [i]) {

    visited local [i] = 1;
    queue [rear++] = i;
    }
  }
 }
```

```
return root;
}
void inorder (struct node * root){
    if(root == NULL) return;
    inorder (root->left);
    printf("%d", root->data);
    inorder(root->right);
}

void preorder (struct node * root) {
    if(root == NULL) return;
    printf("%d", root->data);
    preorder (root->left);
    preorder (root->right);
    printf("%d", root->data);
}
int main() {
    struct node * root = NULL
    int n, value;
    printf("Enter number of node:");
    scanf("%d", &n);
    printf("Enter %d", &value);
    root = insert (root, value);
}
```

```
Printf("\n In - order - traversal :");
    inorder (root);
Printf("\n pre - order transversal :");
    preorder (root);
Printf("\n post - order transversal :");
    post order (root);
        Printf("\n");
        return 0;
        }
```

Sample output :

input :

Enter number of node : 7

Enter 7 values : 50 30 70 20 40 60 80

output :

In - order traversal : 20 30 40 50 60 70 80

pre - order traversal : 50 30 20 40 70 60 80

post - order traversal : 20 40 30 60 80 70 50

③ Stack .o using arrays:

1. Push.c)

Program:

```
# include <stdio.h>
# define max 5.
   int stack [max];
   int top = -1;
    void push (int value)
   {
    Printf( "Stack overflow . \n");
    }
     else
   {
    top++;
     Stack [top] = value;
   Printf("%d inserted into Stack \n", value);
  }
}
int main ()
{
  Push (10);
    Push (20);
      Push (30);
   Printf ("\n stack element are :\n");
```

```
for (int i = top ; i >= 0 ; i--)
{
printf("%d\n", stack[i]);
}
return 0;
```

output:

Enter the number of elements to Push : 3

Enter values : 10

10 pushed into stack

Enter the value : 20

20 pushed into stack

Enter vale : 30

30 pushed into stack

2. POP)

Programme :

```
# include <stdio.h>

# define MAX 5;

int stack [MAX] = {10, 20, 30};

    int top = 2;

    void POP()

{

if (top == -1)

{

  printf("Stack underflow \n");

  }

else

{ printf("%d popped from stack \n", stack[top]);

   top--;

  }

}

int main()

  {

   printf("Stack before pop :\n");

  for(int i= top ; i>=0 ; i--)

  {

   Printf("%d \n", stack[i]);

    }

    POP();
```

```
printf("\n stack after pop :\n");
for (int j = top ; j>=0; j--)
{
printf("%d\n", stack[j]);
}
return 0;
}
```

output :

stack before pop

30
20
10

Deleted element : 30

stack after pop :

20
10

3. peek.c)

Program :

```
# include <stdio.h>
# define MAX 5
int stack [MAX] = { 10, 20, 30 };
    int top = 2;
     void peek ()
    {
      if (top == -1)
      {
        printf ("stack is empty \n");
      }
      else
      {
       print ("Top element is : %d \n", stack
                                    [top]);
      }
}
    int main ()
    {
      printf("stack element are : \n");
     for (int i = top ; i >= 0 ; i --)
      {
        Printf ("%d \n", stack [i]);
      }
      peek ();
```

return 0;

}

output:

Stack elements

30

20

10

Top element:

30